



اُنِيْوَ تِكْنُوْلُوجِيْ مَارَا
UNIVERSITI
TEKNOLOGI
MARA

FACULTY

Faculty of Computer and Mathematical Sciences

PROGRAMME

CS251 Bachelor of Computer Science (Hons.)

Netcentric Computing

SUBJECT

CSP600 PROJECT FORMULATION

CHAPTER 1 FINAL YEAR PROJECT

PREPARED FOR:

Prof. Madya Ts. Dr. Alya Geogiana Binti Buja

STUDENT PROFILE:

Student Name:	MUHAMMAD SAIFUL IQBAL BIN ABDUL RANI
Student ID:	2025101407
Group:	CDCS2515A
Area of Study:	Cybersecurity
Suggested Project Title:	VERITAS: Vulnerability Evaluation and Real-time Intelligence Through Automated Simulation.

1. Background Study

The Rise of AI-Assisted "Vibe Coding" and Simplified Development. The landscape of software engineering has undergone a paradigm shift with the emergence of "vibe coding," a phenomenon where individuals utilize Large Language Models (LLMs) to generate entire web applications through natural language prompts. This democratization of development allows creators with minimal formal programming backgrounds to build functional platforms at unprecedented speeds. However, this high-level approach often detaches the developer from the underlying security architecture, as the focus is placed entirely on functionality and "vibe" rather than secure coding standards.

The Inherent Security Risks of AI-Generated Code. While AI-assisted tools are highly efficient, they are frequently trained on vast repositories of code that may contain legacy vulnerabilities or insecure patterns. AI models tend to prioritize fulfilling the user's functional request over implementing robust security protocols like input sanitization or proper authorization. Consequently, developers who lack foundational security knowledge often "copy-paste" AI-generated snippets without realizing they are introducing critical entry points for cyber-attacks into their applications.

The Incompatibility of Expert-Level Security Tools with Novice Developers. Traditional vulnerability scanners and security auditing tools are predominantly designed for cybersecurity professionals and seasoned engineers. These tools produce complex, jargon-heavy outputs and require intricate configurations that are often beyond the technical reach of a novice or AI-dependent developer. This creates a significant barrier where the very individuals who need the most security guidance are unable to use the existing tools meant to provide it, leaving their applications exposed to the "Big 5" OWASP threats.

The Power of Simulation as a Learning and Validation Tool. For a developer who operates primarily through visual and functional results, seeing a vulnerability in action is far more impactful than reading a technical log. Headless browser simulation technology, such as Playwright, offers a unique opportunity to demonstrate security flaws by physically "showing" the exploit happen in a controlled environment. By providing visual evidence through automated screenshots and interaction logs, developers can bridge the gap between abstract AI-generated code and the real-world consequences of an unpatched vulnerability.

Introducing VERITAS: Empowering AI-First Developers through Intelligence. To address these challenges, this project introduces VERITAS (Vulnerability Evaluation and Real-time Intelligence Through Automated Simulation). VERITAS is specifically engineered as a security companion for the modern AI-assisted developer, combining the precision of browser simulation with the clarity of generative AI. By simulating hacker behaviors and utilizing OpenAI to translate technical hazards into actionable, layman-friendly instructions, VERITAS ensures that even developers with low foundational knowledge can build, audit, and secure their applications with confidence.

2. Problem Statement

1. High AI-Dependency Coupled with Low Security Confidence

Empirical data from a preliminary study indicates a profound reliance on AI-assisted development among students and junior developers, with a significant majority frequently utilizing Large Language Models (LLMs) to generate web application code (Muhammad Saiful Iqbal, 2026). However, this high usage does not correlate with security assurance; respondents reported a notable lack of confidence in the security of AI-generated output (Muhammad Saiful Iqbal, 2026). This observation is supported by academic research which identifies that LLMs for code (LLMc) frequently raise critical concerns regarding trustworthiness, often producing code that lacks well-defined trust metrics beyond mere accuracy (Khatri, 2025). This "vibe coding" trend creates a dangerous security gap where development speed is prioritized over the fundamental verification of the generated logic (Muhammad Saiful Iqbal, 2026).

2. Technical Complexity and Inaccessibility of Security Reporting

A critical barrier to secure development is the nature of current security reporting, which is overwhelmingly perceived as too technical for developers lacking a specialized cybersecurity background (Muhammad Saiful Iqbal, 2026). Preliminary survey results show a strong consensus that existing reports are obscure, leading to a disconnect between the detection of a vulnerability and the practical ability to remediate it (Muhammad Saiful Iqbal, 2026). Furthermore, recent studies demonstrate that AI-generated code is often more repetitive and prone to unused constructs compared to human-written programs, which can obscure security flaws from the developer's view (Cotroneo et al., 2025). Without simplified, actionable insights, developers remain unable to interpret complex scanner results, leaving applications exposed to preventable threats.

3. Inefficiency Caused by Frequent False Positives

The effectiveness of automated security validation is further undermined by the high frequency of "False Positives" reported vulnerabilities that do not pose an actual risk in a real-world environment (Muhammad Saiful Iqbal, 2026). Developers frequently encounter these inaccurate findings, which lead to "alert fatigue" and a subsequent loss of trust in automated security tools (Muhammad Saiful Iqbal, 2026). Large-scale empirical comparisons have shown that AI-generated code exhibits different defect patterns than human-written code, often requiring specialized analysis to distinguish between structural complexity and actual security vulnerabilities (Cotroneo et al., 2025). This lack of accuracy forces developers to waste valuable time manually validating theoretical risks rather than addressing confirmed, exploitable threats.

4. Urgent Need for Exploit Validation and Simplified Remediation

There is a critically important requirement among modern developers to differentiate between theoretical risks and vulnerabilities that are "actually exploitable" (Muhammad Saiful Iqbal, 2026). Survey participants indicated that an AI tool capable of safely simulating an attack to provide visual proof of a bug would be exceptionally helpful for validation and learning (Muhammad Saiful Iqbal, 2026). Academic discourse suggests that advanced security analysis frameworks must combine specialized agents such as structural analysis and LLM-enhanced detection to overcome the limitations of traditional security tools (Jiao et al., 2025). Consequently, there is a clear demand for security reports to be transformed from dense technical logs into simplified, evidence-based tools that include visual security scores and actionable fix guidance (Muhammad Saiful Iqbal, 2026).

3. Project Aim and Objective

3.1 Project Aim

The primary aim of VERITAS is to develop a browser-simulation-based security validation system that assists developers regardless of their foundational security knowledge in securing both traditional and AI-generated web applications. The system focuses on performing active exploit confirmation through headless browser simulation to distinguish between theoretical risks and real-world threats, while utilizing Generative AI to provide simplified remediation guidance for developers who are increasingly dependent on AI-assisted coding.

3.2 Research Objectives

To ensure the project follows the S.M.A.R.T. criteria and aligns with the findings of the preliminary study, the following objectives are established:

1. To Design a modular architecture for the VERITAS framework that integrates headless browser simulation (Playwright) with Large Language Models (LLMs) to automate the verification of critical web vulnerabilities.
2. To Develop a functional prototype capable of orchestrating automated "attacker-style" interactions to provide visual proof-of-concept (screenshots) and validate the exploitability of risks in both human-written and AI-generated code.
3. To Evaluate the detection accuracy and reporting effectiveness of the framework by using DVWA and OWASP Juice Shop as technical baselines, followed by testing against AI-generated web application test cases to measure the improvement in developer comprehension.

4. Project Scope

The scope of this research facilitates the systematic development and evaluation of the VERITAS framework. To maintain research integrity and feasibility, the boundaries are categorized into covered domains and explicit exclusions, specifically focusing on the intersection of automated security and AI-assisted development.

4.1 In-Scope: Targeted Research Areas

The research encompasses the following technical and operational components:

Operational Methodologies:

- **Hybrid Security Analysis:** The framework executes a dual-mode analysis: Black Box Mode for endpoint discovery and exploit validation via a URL, and White Box Mode for inspecting uploaded source code to identify insecure patterns in AI-generated snippets.
- **Evidence-Based Validation:** The system utilizes headless browser simulation to physically interact with web elements, capturing visual evidence (screenshots) to confirm successful exploits.

Technical Focus:

- **Vulnerability Classification:** The system identifies and validates the "Big 5" critical vulnerabilities derived from the OWASP Top 10 standard.
- **Simulated Attacker Behavior:** The framework orchestrates automated interactions to mimic human-style hacking, ensuring deep navigation through multi-step application logic.

AI Integration & Developer Support:

- **Simplified Remediation Intelligence:** The AI module translates complex technical logs into layman-friendly explanations specifically for developers with limited foundational security knowledge.
- **Actionable Prompt Generation:** The system generates automated remediation reports that include simplified fixes and security scores to help developers fix vulnerabilities faster.

Evaluation Environments:

- Industry Baselines: The research utilizes DVWA and OWASP Juice Shop as standardized laboratory environments to establish technical accuracy.
- AI-Generated Test Cases: The framework is further evaluated against custom web applications built using AI-assisted tools (e.g., ChatGPT, GitHub Copilot) to test its effectiveness in modern "vibe coding" scenarios.

4.2 Out-of-Scope: Research Exclusions

To ensure the safety, ethical compliance, and focus of the study, the following areas are excluded:

- Production Environments: The system strictly avoids targeting real-world production systems.
- Non-Consensual Testing: The framework mandates explicit authorization and is restricted to controlled environments to ensure ethical adherence.
- Infrastructure Layers: This research excludes network-level security testing (e.g., firewall or port scanning), focusing exclusively on web application layer vulnerabilities.
- Destructive Exploitation: The study omits the use of destructive payloads or "Denial of Service" (DoS) attacks that could result in permanent data loss or server instability.

5. Project Significance

This research is vital because it addresses a critical modern challenge: the increasing vulnerability of web applications built in the era of AI-assisted "vibe coding." While AI-generated code significantly accelerates the development lifecycle, it frequently lacks the security depth required to withstand sophisticated cyber-attacks. VERITAS is essential for bridging the gap between passive vulnerability identification and practical secure development by providing a visual, evidence-based mechanism to confirm whether a theoretical risk is truly exploitable.

The primary implication of this work is the mitigation of "alert fatigue" and "security frustration" among developers. Data from the preliminary study suggests that developers—particularly students and those with less than three years of experience—are often overwhelmed by the technical complexity of traditional security reports. By drastically reducing false-positive results through active browser-based validation, VERITAS allows developers to focus their limited resources on remediating verified, high-impact threats. Furthermore, the integration of AI-assisted remediation allows developers, especially those without a specialized cybersecurity background, to comprehend and fix complex security flaws more efficiently.

This study synergistically links three major domains of knowledge: Software Engineering, Cybersecurity, and Artificial Intelligence. It integrates traditional vulnerability scanning methodologies with active simulation techniques to enhance the modern Software Development Lifecycle (SDLC). By aligning with the OWASP Top 10 standard, the research ensures the framework remains relevant to current industry benchmarks while contributing pioneering data on the security of AI-generated web logic.

As the global digital infrastructure becomes increasingly automated, our understanding of security must evolve from static defenses to adaptive, intelligent validation. This project highlights the necessity of human-readable security intelligence in an era where software is constructed by both humans and machines. It emphasizes that security is not merely about finding bugs, but about understanding the real-world impact of those bugs on the safety and privacy of the digital world.

VERITAS introduces a novel perspective by shifting the security paradigm from "detection-only" to "exploit-confirmation." Instead of overwhelming developers with dense technical logs, this research proposes an adaptive simulation model that proves the existence of a flaw through controlled execution and visual proof. This approach transforms security reports from passive documentation into active, educational tools that foster a deeper understanding of secure coding practices for a new generation of AI-first developers.

References

- Muhammad Saiful Iqbal Abdul Rani (2026). *AI-Assisted Web Development and Security Validation Challenges* [Unpublished survey data]. Faculty of Computer and Mathematical Sciences, Universiti Teknologi MARA (UiTM).
- Cotroneo, D., Improta, C., & Liguori, P. (2025). Human-written vs. AI-generated code: A large-scale study of defects, vulnerabilities, and complexity. *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 252–263.
<https://doi.org/10.1109/ISSRE66568.2025.00035>
- Jiao, R., Zhang, Y., Li, J., & Ma, B. (2025). SCCA: A multi-agent code security analysis framework for AI-assisted code generation. *2025 IEEE 22nd International Conference on Mobile Ad-Hoc and Smart Systems (MASS)*, 549–554.
<https://doi.org/10.1109/MASS66014.2025.00089>
- Khati, D. (2025). Trustworthiness of large language models for code. *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 208–210. <https://doi.org/10.1109/ICSE-Companion66252.2025.00063>